Module 6: Conditional Rendering (React)

Conditional rendering in React works the same way conditions work in JavaScript. You decide **what UI to render based on state, props, or expressions**.

---

1 If Conditions in JSX

◈ Concept

JSX does **not support if statements directly inside JSX**.
Instead, you use:

- if **before** the return
- or assign JSX to a variable

◈ Example

```
function Greeting({ isLoggedIn }) {
  if (isLoggedIn) {
    return <h1>Welcome back!</h1>;
  }
  return <h1>Please log in.</h1>;
}
```

◈ Using Variables

```
function Status({ isOnline }) {
  let message;

  if (isOnline) {
    message = <p>User is online</p>;
  } else {
    message = <p>User is offline</p>;
  }
```

```
  return <div>{message}</div>;
}
```

📝 **Task**

- Create a component that displays **“Admin Panel”** if isAdmin is true, otherwise **“User Dashboard”**.

---

2 Ternary Operators

◈ **Concept**

The **ternary operator** is the most common way to handle simple conditions in JSX.

Syntax:

```
condition ? trueExpression : falseExpression
```

◈ **Example**

```
function LoginStatus({ isLoggedIn }) {
  return (
    <h1>
      {isLoggedIn ? "Welcome User" : "Please Sign In"}
    </h1>
  );
}
```

◈ **With JSX Blocks**

```
function Profile({ hasProfile }) {
  return (
    <div>
      {hasProfile ? <ProfilePage /> : <CreateProfile />}
    </div>
  );
}
```

📝 Task

- Use a ternary operator to display **“Light Mode”** or **“Dark Mode”** based on a theme state.

---

3 Logical && Rendering

◈ Concept

Use && when you want to render something **only if a condition is true**.

If the condition is false, **nothing is rendered**.

◈ Example

```
function Notifications({ count }) {
  return (
    <div>
      {count > 0 && <p>You have {count} notifications</p>}
    </div>
  );
}
```

⚠ Important Note

Avoid rendering numbers directly:

```
{0 && <p>No messages</p>} // renders 0
```

Use explicit conditions:

```
{count > 0 && <p>Messages available</p>}
```

📝 Task

- Show a **warning message** only when batteryLevel < 20.

---

4 Rendering Lists

◈ Concept

React renders lists using JavaScript’s map() method.

◈ Example

```
function FruitsList() {
  const fruits = ["Apple", "Banana", "Mango"];

  return (
    <ul>
      {fruits.map(fruit => (
        <li>{fruit}</li>
      ))}
    </ul>
  );
}
```

◈ Rendering Objects

```
function UsersList() {
  const users = [
    { id: 1, name: "Alice" },
    { id: 2, name: "Bob" }
  ];

  return (
    <ul>
      {users.map(user => (
        <li key={user.id}>{user.name}</li>
      ))}
    </ul>
  );
```

}

📝 **Task**

- Render a list of **products** showing name and price using map().

5⃣ Keys in Lists

🔷 **Concept**

Keys help React identify which items have changed, added, or removed.

Keys must be:

- **Unique**
- **Stable**
- Usually an id

🔷 **Correct Usage**

```
<li key={user.id}>{user.name}</li>
```

❌ **Avoid Using Index as Key (unless no other option)**

```
{items.map((item, index) => (
  <li key={index}>{item}</li>
))}
```

🔷 **Why Keys Matter**

- Improves performance
- Prevents incorrect UI updates
- Helps React efficiently re-render

📝 **Task**

- Create a list of **tasks (todo items)** using unique keys from an id.

Task 1: User Dashboard with Role-Based Access (Advanced)

Scenario

Build a **User Dashboard** that changes its UI based on:

- Login status
- User role (admin / user)
- Notification count

Requirements

1. If the user is **not logged in**, show:
    - “Please log in to continue”
2. If logged in:
    - Display **Welcome message**
    - If role is admin, show **Admin Panel**
    - Else show **User Panel**
3. Show notification message **only if notifications > 0**
4. Render a list of notifications with unique keys

Module 7: React Hooks

Hooks let you **use React features (state, lifecycle, context)** inside **function components**, without writing class components.

---

1 What Are Hooks?

◈ Concept

Hooks are **special functions** that let function components:

- Store state
- Run side effects
- Access context
- Share logic

◈ Rules of Hooks

✓ Call Hooks **only at the top level**
✓ Call Hooks **only inside React function components or custom Hooks**

◈ Example

```
import { useState } from "react";

function Counter() {
  const [count, setCount] = useState(0);

  return (
    <button onClick={() => setCount(count + 1)}>
      Count: {count}
    </button>
  );
}
```

📝 Task

- Convert a class component with state into a **function component using Hooks**.

---

2 □ useState Hook

◈ Concept

useState lets a component **remember values** between renders.

◈ Syntax

```
const [state, setState] = useState(initialValue);
```

◈ Example

```
function Form() {
  const [name, setName] = useState("");

  return (
    <input
```

```
      value={name}
      onChange={e => setName(e.target.value)}
      placeholder="Enter name"
    />
  );
}
```

📝 **Task**

- Create a **toggle button** that switches between ON and OFF.

---

3 useEffect Hook

◈ **Concept**

useEffect handles **side effects** such as:

- Fetching data
- Subscriptions
- Timers
- DOM updates

◈ **Syntax**

```
useEffect(() => {
  // side effect
}, [dependencies]);
```

◈ **Example (Runs Once on Mount)**

```
import { useEffect } from "react";

function PageTitle() {
  useEffect(() => {
    document.title = "React Hooks";
```

```
  }, []);

  return <h1>Welcome</h1>;
}
```

📝 **Task**

- Fetch user data from an API when the component mounts.

4⃣ Component Lifecycle (Basic)

🔷 **Concept**

Hooks replace class lifecycle methods:

Class Lifecycle	Hook Equivalent
componentDidMount	useEffect(() => {}, [])
componentDidUpdate	useEffect(() => {}, [deps])
componentWillUnmount	Cleanup function

🔷 **Example**

```
useEffect(() => {
  console.log("Component mounted");

  return () => {
    console.log("Component unmounted");
  };
}, []);
```

📝 **Task**

- Log messages for **mount and unmount** phases.

5⃣ Dependency Array

◈ Concept

The dependency array controls **when useEffect runs**.

◈ Behavior

Dependency Array	Effect Runs
Not provided	On every render
[]	Once (on mount)
[value]	When value changes

◈ Example

```
function Counter() {
  const [count, setCount] = useState(0);

  useEffect(() => {
    console.log("Count changed:", count);
  }, [count]);

  return <button onClick={() => setCount(count + 1)}>+</button>;
}
```

📝 Task

- Trigger an effect **only when a theme value changes**.

---

6 □□Cleanup Functions

◈ Concept

Cleanup functions prevent:

- Memory leaks
- Unwanted subscriptions
- Multiple timers

They run:

- Before the effect re-runs
- When the component unmounts

◈ **Example (Timer Cleanup)**

```
useEffect(() => {
  const timer = setInterval(() => {
    console.log("Running...");
  }, 1000);

  return () => clearInterval(timer);
}, []);
```

📝 **Task**

- Create a timer that **starts on mount and stops on unmount**.

---

7 □ useContext Hook

◈ **Concept**

useContext allows data to be shared **without prop drilling**.

◈ **Example**

```
import { createContext, useContext } from "react";

const ThemeContext = createContext();

function ThemeProvider() {
  return (
    <ThemeContext.Provider value="dark">
      <Toolbar />
```

```
    </ThemeContext.Provider>
  );
}

function Toolbar() {
  const theme = useContext(ThemeContext);
  return <p>Theme: {theme}</p>;
}
```

📝 **Task**

- Create a **language context** and consume it in a nested component.

Build a **Theme Switcher App**:

- Use useState for theme
- Use useContext to share theme
- Use useEffect to update page title
- Add cleanup logic for event listeners